

## TD Intelligence Artificielle n° 1

## Recherche dans un espace d'états

## Exercice 1 Traversée du pont

Quatre personnes doivent traverser un pont en moins de 17 minutes. Chacune d'entre elles marche à une vitesse maximale donnée. L'une peut traverser le pont en 1 minute, l'autre en 2 minutes, l'autre en 5 minutes, et la dernière en 10 minutes. Il est impossible de traverser le pont sans torche, et le groupe n'en a qu'une. Enfin, le pont ne peut supporter que le poids de 2 individus.

On cherche à déterminer l'ordre dans lequel les quatre personnes doivent traverser.

1. Choisissez un formalisme permettant de représenter les états possibles.
2. Identifiez, dans ce formalisme, l'état initial et l'ensemble des états finaux.
3. Définissez les opérateurs avec leur coût.
4. Quel algorithme trouvera la solution optimale ?

## Exercice 2 Problème du taquin

Le taquin est un casse-tête dont le principe est de remettre en ordre des tuiles numérotées. Le jeu se présente sous la forme d'une grille remplie de pièces coulissantes, qui peuvent être déplacées grâce à la présence d'une case vide. Partant d'une situation où les pièces sont mélangées, l'objectif est donc d'atteindre une configuration but dans laquelle chaque tuile est à sa place.

Ici, on considère une version à huit pièces du jeu, en cherchant à réaliser la transformation illustrée par la figure 1.

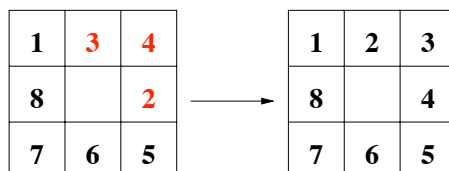


FIGURE 1 – Une situation initiale (à gauche) et la configuration but (à droite) pour la version à huit pièces du taquin.

1. Choisissez un formalisme permettant de représenter les états possibles.
2. Combien d'états possibles y a-t-il ?
3. Identifiez, dans ce formalisme, l'état initial et l'ensemble des états finaux.
4. Proposez trois heuristiques pour résoudre le jeu de taquin avec l'algorithme A\*.
5. Appliquez l'algorithme A\* avec la meilleure de ces heuristiques.

## Exercice 3 Touriste en Roumanie

On considère un touriste en fin de vacances en Roumanie, qui cherche à rejoindre l'aéroport situé à Bucharest par le plus court chemin. On considère la carte (simplifiée) de la Roumanie de la figure 2 et on dispose par ailleurs d'informations supplémentaires résumées dans le tableau 1.

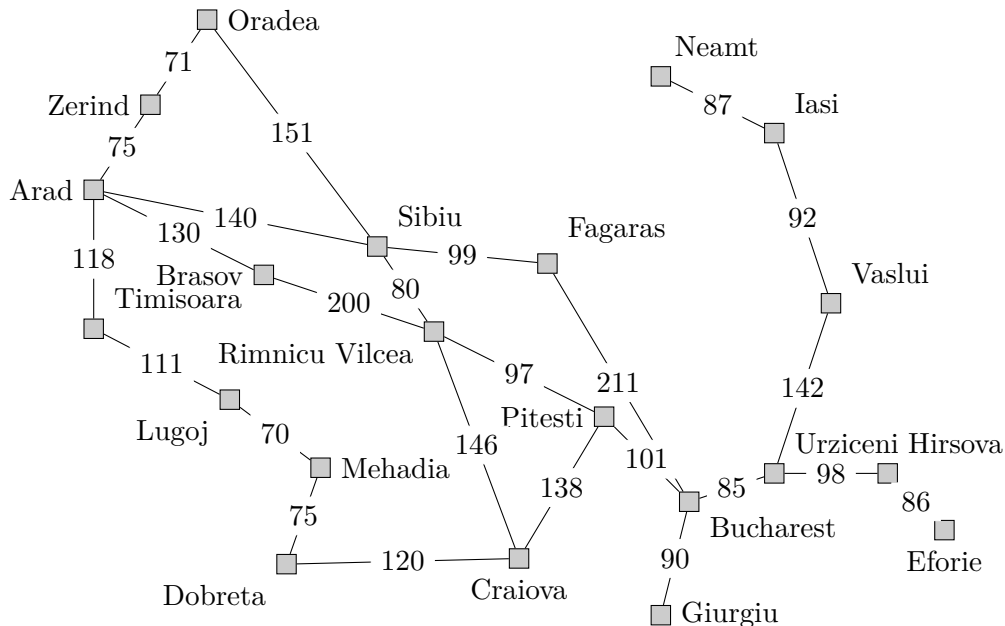


FIGURE 2 – Carte routière simplifiée de la Roumanie, avec longueur des routes (km).

Pour les questions de recherche d'itinéraire ci-dessous, vous dessinerez l'arbre de recherche en indiquant le numéro de l'étape lorsqu'un nœud est choisi dans la frontière. Dans le cas où le chemin menant à un état déjà rencontré serait remis en cause, dessinez un nouveau nœud et barrez l'ancien.

1. Appliquez l'algorithme de Dijkstra pour trouver un itinéraire allant d'Arad à Bucharest.
2. Proposez une heuristique  $h$  pour le problème du touriste (autre que  $h_0 = 0$ ). Est-elle minorante (c.-à-d. admissible) ? Justifiez en une phrase.
3. Appliquez l'algorithme A\* pour trouver un itinéraire allant d'Arad à Bucharest.
4. Appliquez l'algorithme de recherche gloutonne (c.-à-d. gourmande) pour aller d'Arad à Bucharest.

## Annexe

|           |     |         |     |                |     |           |     |
|-----------|-----|---------|-----|----------------|-----|-----------|-----|
| Arad      | 366 | Fagaras | 176 | Mehadia        | 241 | Sibiu     | 253 |
| Brasov    | 254 | Giurgiu | 77  | Neamt          | 234 | Timisoara | 329 |
| Bucharest | 0   | Hirsova | 151 | Oradea         | 380 | Urziceni  | 80  |
| Craiova   | 160 | Iasi    | 226 | Pitesti        | 100 | Vaslui    | 199 |
| Dobreta   | 242 | Lugoj   | 244 | Rimnicu Vilcea | 193 | Zerind    | 374 |
| Eforie    | 161 |         |     |                |     |           |     |

TABLE 1 – Distance à vol d'oiseau jusqu'à Bucharest (km).

## Annexe

---

### Algorithme 1 Dijkstra

---

**Entrée :**

$s_{\text{init}}$  : l'état initial du problème

**Sortie :**

- une solution, c.-à-d. un chemin entre  $s_{\text{init}}$  et un état final  
ou

- ECHEC, s'il n'existe pas de solution

**fonction** rechDijkstra( $s_{\text{init}}$ )

$DejaDev \leftarrow \emptyset$

$Frontiere \leftarrow \{s_{\text{init}}\}$  /\* en général on implémente une file de priorité \*/

$g[s_{\text{init}}] \leftarrow 0$

**tant que**  $Frontiere \neq \emptyset$  **faire**

$s \leftarrow \text{choixGMin}(Frontiere)$

**si** estFinal( $s$ ) **alors**

**retourner** construireSolution( $s, pere$ )

**sinon**

$Frontiere \leftarrow \text{supprimer}(s, Frontiere)$

$DejaDev \leftarrow \text{ajouter}(s, DejaDev)$

**pour tout**  $s' \in \text{successeurs}(s)$  **faire**

**si**  $s' \notin DejaDev$  **et** ( $s' \notin Frontiere$  **ou**  $g[s] + \text{cout}(s, s') < g[s']$ ) **alors**

/\* soit  $s'$  n'a jamais été rencontré avant, soit  $s'$  est dans la frontière et le nouveau chemin qui y mène coûte moins que l'ancien \*/

$pere[s'] \leftarrow s$

$g[s'] \leftarrow g[s] + \text{cout}(s, s')$

$\text{ajouter}(s', Frontiere)$

**fin si**

**fin pour**

**fin si**

**fin tant que**

/\* pas de solution \*/

**retourner** ECHEC

**fin fonction**

---

**Remarques :**

- Les chemins menant aux états dans  $DejaDev$  ne sont jamais remis en cause (c.-à-d. ils sont optimaux).
- Si tous les arcs ont un coût identique (p.ex. 1), l'algorithme de Dijkstra est équivalent à la recherche en largeur.

---

**Algorithme 2** A/A\***Entrée :**

- $s_{\text{init}}$  : l'état initial du problème
- $h$  : une fonction heuristique (si  $h$  est minorante, l'algorithme devient A\*)

**Sortie :**

- une solution, c.-à-d. un chemin entre  $s_{\text{init}}$  et un état final  
ou
- ECHEC, s'il n'existe pas de solution

**fonction** rechA( $s_{\text{init}}, h$ ) $DejaDev \leftarrow \emptyset$  $Frontiere \leftarrow \{s_{\text{init}}\}$  /\* en général on implémente une file de priorité \*/ $g[s_{\text{init}}] \leftarrow 0$  $f[s_{\text{init}}] \leftarrow h(s_{\text{init}})$ **tant que**  $Frontiere \neq \emptyset$  **faire** $s \leftarrow \text{choixFMin}(Frontiere)$ **si** estFinal( $s$ ) **alors****retourner** construireSolution( $s, pere$ )**sinon** $Frontiere \leftarrow \text{supprimer}(s, Frontiere)$  $DejaDev \leftarrow \text{ajouter}(s, DejaDev)$ **pour tout**  $s' \in \text{successeurs}(s)$  **faire****si**  $s' \notin DejaDev \cup Frontiere$  **ou**  $g[s] + \text{cout}(s, s') < g[s']$  **alors**/\* soit  $s'$  n'a jamais été rencontré avant, soit le nouveau chemin qui mène à  $s'$   
coûte moins que l'ancien \*/ $pere[s'] \leftarrow s$  $g[s'] \leftarrow g[s] + \text{cout}(s, s')$  $f[s'] \leftarrow g[s'] + h(s')$ **si**  $s' \in DejaDev$  **alors** $\text{supprimer}(s', DejaDev)$ **fin si** $\text{ajouter}(s', Frontiere)$ **fin si****fin pour****fin si****fin tant que**

/\* pas de solution \*/

**retourner** ECHEC**fin fonction**

---

**Remarques :**

- Au contraire de ce qui se passe dans l'algorithme de Dijkstra, les chemins menant aux états dans  $DejaDev$  peuvent être remis en cause (c.-à-d. ils ne sont pas nécessairement optimaux, sauf le chemin solution renvoyé à la fin).
- $h = 0$  est une heuristique trivialement minorante (on ne considère que des coûts positifs), mais pas informative. Elle ramène A\* à l'algorithme de Dijkstra :  $f = g + h = g$ .